

プログラミングI 演習

Computer Programming I: Exercises

第8回 関数と引数

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

1

本日の講義

本日の目標

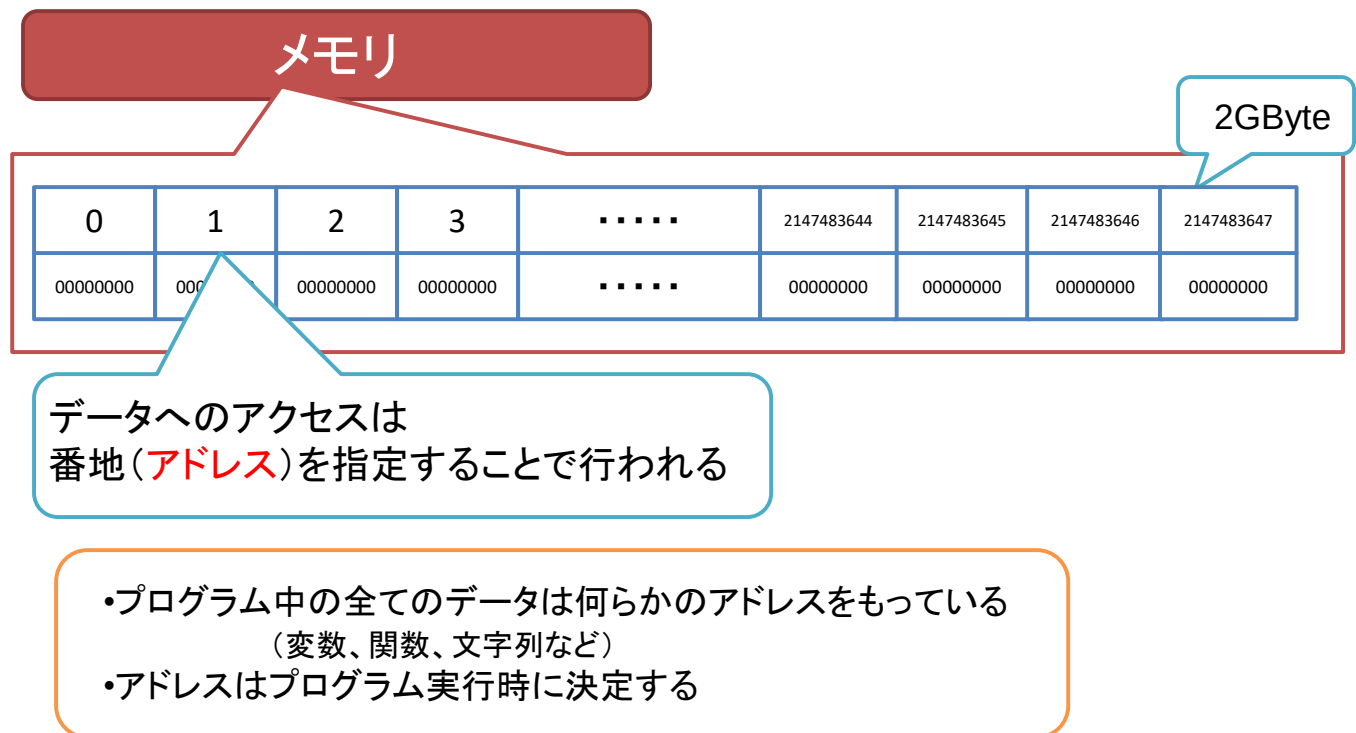
- 関数の役割を知ろう(再掲)
- 関数の引数として配列を渡せるようになろう

本日の資料

1. 前回のおさらい
2. 配列を引数とする関数
3. 演習課題

1. 前回のおさらい

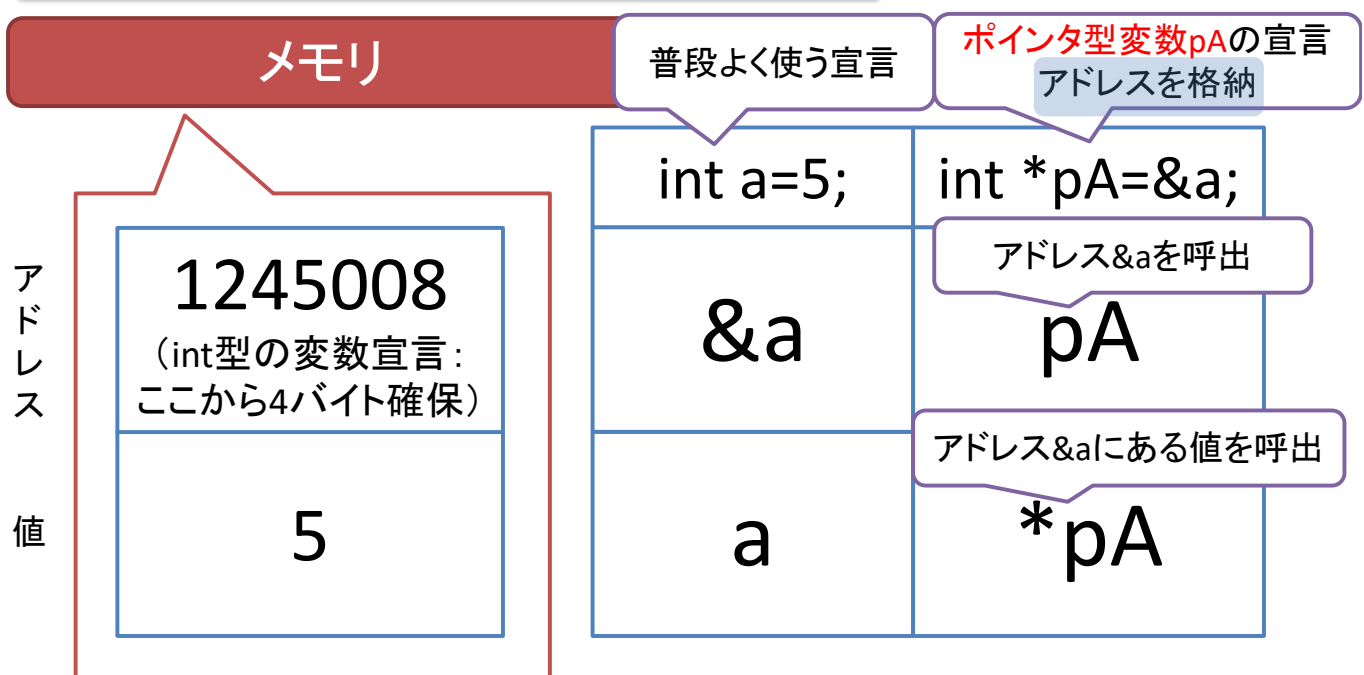
メモリの構造



3

1. 前回のおさらい

■ポインタ型(プログラム内での表現)



4

1. 前回のおさらい

アドレス	&a	=	pA
値	a		*pA

■ポインタ型変数

- アドレスを保存するための変数
- メモリ上のアドレスを利用したデータのやりとりができる

ポインタを使ったプログラムの例 (※実際に動かすときには#include<stdio.h>を付けること)

```
int main(void){
    int a=5;
    int *pA;
    pA = &a;
    printf("変数aの値は%dです。¥n", a);
    printf("変数aのアドレスは%p = %luです。¥n", &a, (unsigned
        long int)&a);
    printf("ポインタpAの値は%p = %luです。¥n", pA, (unsigned
        long int)pA);
    printf("ポインタpAが示すアドレスにある値は%dです。¥n", *pA);
    return 0;
}
```

実行結果 (※aのアドレスやpAの値は環境によって異なる)
変数aの値は5です。
変数aのアドレスは0x7ffff1c2debc = 140737249468092です。
ポインタpAの値は0x7ffff1c2debc = 140737249468092です。
ポインタpAが示すアドレスにある値は5です。

(%lu : unsigned long intの指定子)

5

1. 前回のおさらい

アドレス	&a	=	pA
値	a		*pA

■ポインタ型変数

ポインタ型の変数「pA」の宣言

ポインタ型では、そのアドレスに「何型の変数」があるのかを識別するために

「 **型名** * **ポインタ変数名** ; 」

の形式で宣言します

```
int main(void){
    int a=5;
    int *pA;
    pA = &a;
    printf("変数aの値は%dです。¥n", a);
    printf("変数aのアドレスは%p = %luです。¥n", &a, (unsigned
        long int)&a);
    printf("ポインタpAの値は%p = %luです。¥n", pA, (unsigned
        long int)pA);
    printf("ポインタpAが示すアドレスにある値は%dです。¥n", *pA);
    return 0;
}
```

実行結果 (※aのアドレスやpAの値は環境によって異なる)
変数aの値は5です。
変数aのアドレスは0x7ffff1c2debc = 140737249468092です。
ポインタpAの値は0x7ffff1c2debc = 140737249468092です。
ポインタpAが示すアドレスにある値は5です。

(%lu : unsigned long intの指定子)

6

1. 前回のおさらい

アドレス	&a	pA
値	a	*pA

■ポインタ型変数

アドレスは整数値(%pはアドレスの16進数表示)

「*ポインタ変数名」は、そのポインタが指すアドレスに格納されているデータ(値)を表現する。

```
int main(void){
    int a=5;
    int *pA;
    pA = &a;
    printf("変数aの値は%dです。¥n", a);
    printf("変数aのアドレスは%p = %luです。¥n", &a, (unsigned long int)&a);
    printf("ポインタpAの値は%p = %luです。¥n", pA, (unsigned long int)pA);
    printf("ポインタpAが示すアドレスにある値は%dです。¥n", *pA);
    return 0;
}
```

実行結果(※aのアドレスやpAの値は異なる)

変数aの値は5です。
変数aのアドレスは0x7ffff1c2deb...7249468092です。
ポインタpAの値は0x7ffff1c2deb...37249468092です。
ポインタpAが示すアドレスにある値は5です。

(%lu : unsigned long intの指定子)

7

1. 前回のおさらい

演習7-3 ポインタ変数を用いたプログラムの作成

main関数内に変数aとbを宣言し、aとbの値をそれぞれa/bの商と余りに変更する、void型の関数divAndRemainderを実装せよ。ただし、aとbの値は整数である。

【禁止事項】

- ✕ main関数内でのaとbへの代入
- ✕ 式による出力(変数単体で出力しなければならない)
例) printf("%d %d¥n", a/b, a%b); ← これはダメ

演習7-2ではaとbの値を交換する関数swap1を使いました。
それを上記を満たすように、うまく書き換えてみよう

1. 前回のおさらい

演習7-3 解答例

```
#include<stdio.h>
void divAndRemainder(int *a, int *b){
    int tmp = *a;
    *a = tmp / *b;
    *b = tmp % *b;
}
int main(void){
    int a, b;
    printf("aの値は:"); scanf("%d", &a);
    printf("bの値は:"); scanf("%d", &b);

    divAndRemainder(&a, &b);

    printf("aの値(商)は%d, bの値(余り)は%d¥n", a, b);
    return 0;
}
```

変数tmpを使うと
簡単に実現可

ポインタ渡し

9

2. 配列を引数とする関数

例題: 口座に利子を振り込む関数

口座残高のリストは
次の配列に入っている

```
int Account[100];
```

添え字	残高
0	12345678
1	2345678
2	1234567
.....	
97	345678
98	5678
99	1234567890

作成する関数は.....

引数に配列と利子率を受け取ればよさそう



結果は配列に入れて返せばいいか.....

```
int[100] risi(int LastYear[100],double Interest)
{
    int NextYear[100],i;
    for(i=0;i<100;i++)
        NextYear[i]=LastYear[i]*(1.0+Interest);
    return NextYear;
}
```

10

2. 配列を引数とする関数

例題: 口座に利子を振り込む関数

口座残高のリストは
次の配列に入っている

```
int Account[100];
```

添え字	残高
0	12345678
1	2345678
2	1234567
.....	
97	345678
98	5678
99	1234567890

作成する関数は.....

引数に配列と利子率を受け取ればよさそう



結果は配列に入れて返せばいいか.....

```
int[100] risi(int LastYear[100],double Interest)
{
    int NextYear[100],i;
    for(i=0;i<100;i++)
        NextYear[i]=LastYear[i]*(1.0+Interest);
    return NextYear;
}
```

Cでは(1個の)変数の時と同じようにreturn
文を使って配列を返すことはできない

11

2. 配列を引数とする関数

解答: 口座に利子を振り込む関数

```
void risi(int List[100],double Interest)
{
    int i;
    for(i=0;i<100;i++)
        List[i]=List[i]*(1.0+Interest);
}
```

```
int main(void)
{
    int Account [100];
    .....
    risi(Account,0.003);
    .....
}
```

引数で受け取った配列List
を更新するだけでmainの
Accountも更新(※理由は
後述)

関数に渡すのは配列名(同名のポインタ)
(→つまりポインタ渡し)

```
int Account [100];
```

添え字	残高
0	12345678
1	2345678
2	1234567
.....	
97	345678
98	5678
99	1234567890

12

2. 配列を引数とする関数

配列の作成とメモリ

```
int Account [100];
```

という配列の宣言時にどのようなことが起きるのか



メモリを100個、連続的に確保する

ホテルの部屋を並びで100部屋予約してしまうようなもの

Program A「すいません。100部屋予約したいんですが？」
OS「かしこまりました。では……」

メモリの管理台帳

メモリ番号	ご予約名
1199	Program X
1200	空室
1201	空室
.....	
1298	空室
1299	空室
1300	空室

13

2. 配列を引数とする関数

配列の作成とメモリ

```
int Account [100];
```

という配列の宣言時にどのようなことが起きるのか



メモリを100個、連続的に確保する

ホテルの部屋を並びで100部屋予約してしまうようなもの

Program A「すいません。100部屋予約したいんですが？」
OS「かしこまりました。では……」
OS「1200号室から1299号室まで100部屋お取りしました。」
Program A「ありがとうございます。」

メモリの管理台帳

メモリ番号	ご予約名
1199	Program X
1200	Program A
1201	Program A
.....	
1298	Program A
1299	Program A
1300	空室

メモ: Accountの部屋
1200号室から100部屋分



コンパイラはこのメモを保存しておく

14

2. 配列を引数とする関数

配列の引数の処理

```
void risi(int List[100],double Interest)
{
    int i;
    for(i=0;i<100;i++)
        List[i]=List[i]*(1.0+Interest);
}

int main(void)
{
    int Account [100];
    .....
    risi(Account,0.003);
    .....
}
```

関数の引数が配列名の場合.....

メモ: Listの部屋

〇〇〇号室から100部屋分

まずメモ用紙の雛型が作られる

15

2. 配列を引数とする関数

配列の引数の処理

```
void risi(int List[100],double Interest)
{
    int i;
    for(i=0;i<100;i++)
        List[i]=List[i]*(1.0+Interest);
}

int main(void)
{
    int Account [100];
    .....
    risi(Account,0.003);
    .....
}
```

関数の引数が配列名の場合.....

メモ: Listの部屋

1200号室から100部屋分

メモ内容のコピーが行われる

メモ: Accountの部屋

1200号室から100部屋分

16

2. 配列を引数とする関数

配列の引数の処理

```
void risi(int List[100],double Interest)
{
    int i;
    for(i=0;i<100;i++)
        List[i]=List[i]*(1.0+Interest);
}

int main(void)
{
    int Account [100];
    .....
    risi(Account,0.003);
    .....
}
```

関数の引数が配列名の場合……

メモ: Listの部屋

1200号室から100部屋分

メモ内容のコピーが行われる

プログラム中の「List[i]」は
このメモをもとに
「(1200 + i)号室 のデータ」
というように処理される



ポインタ渡しによりListの部屋(メモリ上の位置)はAccountの部屋と一致し、Listの部屋の中身(データ)の書き換えがAccountの部屋にも反映される

17

ここから演習課題です

18

3. 演習課題

演習8-1 変数の局所性(値渡し)の確認

```
#include<stdio.h>

int Add(int x, int y){
    x = x + y;
    return x;
}

int main(void)
{
    int x=1, y=2;
    printf("xは%d, yは%d\n", x, y);
    printf("x+yは%d\n", Add(x, y));
    printf("xは%d\n", x); /* x+yの結果は入らない */
    return 0;
}
```

このとおりプログラムし実行してみよう。
関数Add(x, y)の処理が終わった後の
printf文の結果を確認しよう。

19

3. 演習課題

演習8-2 配列を引数とする関数(ポインタ渡し)の確認①

```
#include<stdio.h>

int Add(int x[1], int y){
    x[0] = x[0] + y;
    return x[0];
}

int main(void)
{
    int x[1]={1}, y=2;
    printf("xは%d, yは%d\n", x[0], y);
    printf("x+yは%d\n", Add(x, y));
    printf("xは%d\n", x[0]); /* x+yの結果が入る */
    return 0;
}
```

このとおりプログラムして実行してみよう。
関数Add(x, y)の処理が終わった後の
printf文の結果を確認しよう。

20

3. 演習課題

演習8-3 配列を引数とする関数(ポインタ渡し)の確認②

```
#include<stdio.h>
void func(int x[10]){
    int i;
    for(i=0; i<=9; i++){
        x[i] = 9-i;
    }
}
int main(void)
{
    int i, x[10]={0,0,0,0,0,0,0,0,0,0};
    func(x);
    for(i=0; i<=9; i++){
        printf("xの%d番目の要素は%d\n", i, x[i]);
    }
    return 0;
}
```

このとおりプログラムし実行してみよう。
配列を引数とする関数の場合、return
しなくても値が更新されることを確認し
よう。

21

3. 演習課題

演習8-4 ベクトル演算

- [1] 整数 k と一次元配列 a (大きさが n のベクトル、各成分は整数)を引数とし、配列 a をスカラー倍(k 倍)する関数を定義せよ

例

```
void Scalar(int a[10], int k)
```

- [2] 一次元配列 a と一次元配列 b (どちらも大きさが n のベクトル、各成分は整数)を引数とし、 $a+b$ を行う関数を定義せよ

例

```
void AddVector(int a[10], int b[10])
```

22

3. 演習課題

発展8-1 行列演算

- [1] 二次元配列 a と二次元配列 b (どちらも $m \times n$ のサイズの行列. 各成分は整数)を引数とし, $a+b$ を行う関数を定義せよ

例

```
void AddMatrix(int a[5][10], int b[5][10])
```

- [2] 二次元配列 a と二次元配列 b (どちらも $n \times n$ のサイズの行列. 各成分は整数)を引数とし, $a \times b$ を行う関数を定義せよ

例

```
void MultiMatrix(int a[5][5], int b[5][5])
```

23

3. 演習課題

- 発展8-2 血液型調査プログラム (※難易度はかなり高めです. 完成できなくても, 関数の機能を考えたり, 大まかなプログラムを考えるだけでも力になります)

データ保存用の配列を使う

```
int BloodTypeList[100];
```

添え字	血液型
0	2
1	1
2	3
.....	
97	0
98	4
99	1

要求される機能

1. 情報の登録/修正
2. 指定した型の人番号を一覧表示
3. 統計(何型が何人いる)情報の表示



それぞれの処理を行う関数を実装しよう

A型=1、B型=2、O型=3、AB型=4
不明&未調査=0

24

3. 演習課題

発展8-2 血液型調査プログラム

```
#define DATA_SIZE 100
int main(void){
```

データ用配列の宣言

```
    int BloodTypeList[DATA_SIZE];
    int MenuID=0;
    while(MenuID!=4)
    {
        printf("¥n血液型調査管理プログラム¥n¥n");
        printf("1: 登録・修正¥n");
        printf("2: 型別リスト¥n");
        printf("3: 統計情報¥n");
        printf("4: 終了¥n");
        scanf("%d",&MenuID);
```

メインメニューのループ
変数「MenuID」
へ番号を入力

```
        system("reset");/*コンソール画面をクリア*/
```

```
        if(MenuID==1) Regist(BloodTypeList);/*登録/修正関数*/
        else if(MenuID==2) TypeList(BloodTypeList);/*型別リスト関数*/
        else if(MenuID==3) State(BloodTypeList);/*統計情報関数*/
```

各関数を完成
させましょう

```
    }
    return 0;
}
```

注: 先頭に以下のinclude文を忘れずに
#include <stdlib.h>
#include <stdio.h>

25

3. 演習課題

発展8-2 血液型調査プログラム

```
void Regist(int List[DATA_SIZE])
{
    int ID,Type;
    /*リストの指定した場所(ID)にTypeを代入*/
}
```

代入処理を追加しましょう。

安全なプログラム作成のために

配列の添え字などの有効範囲が決まっているものは
これから処理する**数値の範囲チェック**を行う

添え字(ID)の範囲チェック

```
int checkID(int ID)
{
    if( ID < 0 || ID >= DATA_SIZE)return -1;
    return 0;
}
```

血液型(Type)のチェック

```
int checkType(int Type)
{
    if(Type < 0 || Type > 4)return -1;
    return 0;
}
```

3. 演習課題

発展8-2 血液型調査プログラム

※コメント文に従ってプログラムを完成させましょう
指定した型の番号の一覧表示

```
void TypeList(int List[DATA_SIZE])
{
    int Type;
    /*血液型(Type)の入力*/
    /*List[0~99]をループで検索し、該当者を表示*/
}
```

実行例

A型の人
は
2番
8番
13番
15番
です。

統計情報の表示

```
void State(int List[DATA_SIZE])
{
    int TypeNum[5]; /*5種類の型の数を数える*/
    /*List[0~99]をループで検索し、それぞれの  
型から配列「TypeNum」を加算*/
    /*集計結果を表示*/
}
```

実行例

A型	:42人
B型	:18人
O型	:29人
AB型	:5人
不明	:6人

27

3. 演習課題

発展8-2 血液型調査プログラム

100件分のデータを手動で入力するのは大変



乱数を利用

```
#include <stdlib.h> /*rand関数に必要なヘッダー*/
#define DATA_SIZE 100
void main(void)
{
    int BloodTypeList[DATA_SIZE];
    srand(0); /*引数を変えると乱数が変わる*/
    for(i=0;i<DATA_SIZE;i++)
        BloodTypeList[i] = rand()%5;
    /*5で割った余り=0~4の乱数となる*/
}
```

•rand関数(第12回で詳しく解説)

- 0~2147483647までのランダムな整数を返す
- 「srand(seed値)」で乱数列を変更できる

28

本日の説明内容はおしまいです。
課題に取り組んでください。